

ANY OBJECT FINDER

DOCUMENTATION v1.0

First of all, thank you for purchasing this asset through Unity Assets Store. This documentation will guide you through the usage in order for this tool to work properly in your unity project.

WHAT IS ANY OBJECT FINDER?	1
FEATURES	2
1. Search Methods	2
2. Targets (Can select multiple):	4
3. Type	4
4. Location	4
5. Traversal Mode	5
6. Field Name	5
CUSTOMIZATION	5
SUPPORT	7

WHAT IS ANY OBJECT FINDER?

Any Object Finder is a tool that will help you search any unity object, which includes components, prefabs, assets, and literally any object derived from **UnityEngine.Object**, in **Scenes** and/or **Project Folders** (This includes Assets and Packages folders).

Search them by **matching a value condition** to all object fields/members recursively. This also searches through arrays and serializable class instances. If you use Odin Serializer, then it will also search through dictionaries.

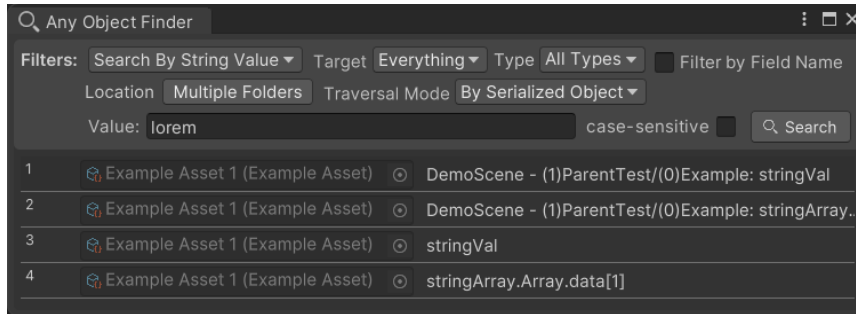
You can search by string, integer, float, boolean, enum, a unity object reference, or a custom condition you can extend by script. Use the advanced filters to narrow down the scope.

The results will be shown in a list containing the object references, and the field name / path. For scene or prefab objects, the GameObject path in the hierarchy will also be shown.

FEATURES

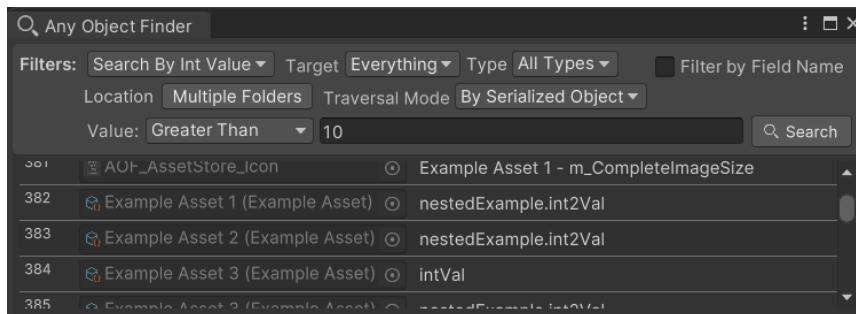
1. Search Methods

a. Search by String Value



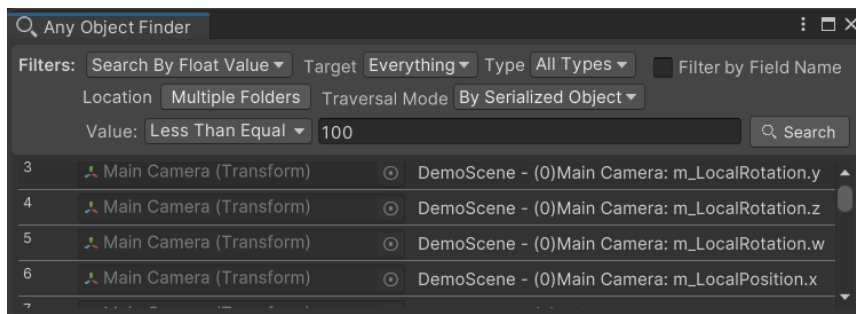
Search in all filtered objects' serialized property where it's a string with the matching text input. The argument is a string value, specifying whether it's case-sensitive or not.

b. Search by Int Value

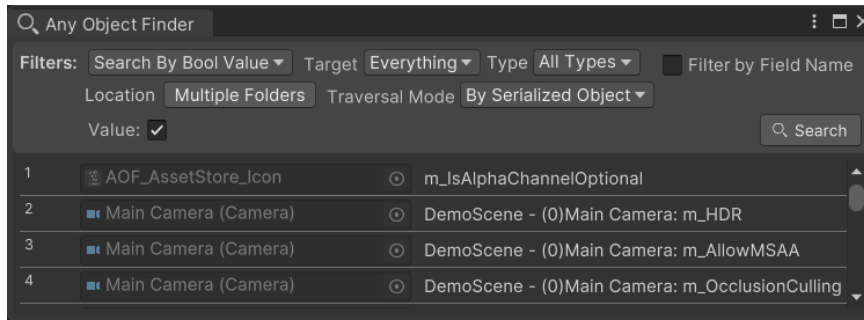


Search in all filtered objects' serialized property where it's an integer with the matched condition. The arguments include the condition type (Equal, Less Than, Less Than Equal, More Than, More Than Equal, Not Equal) and an integer value to compare.

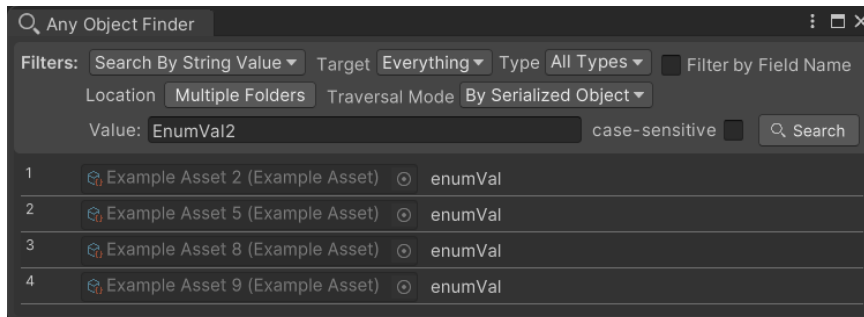
c. Search by Float Value



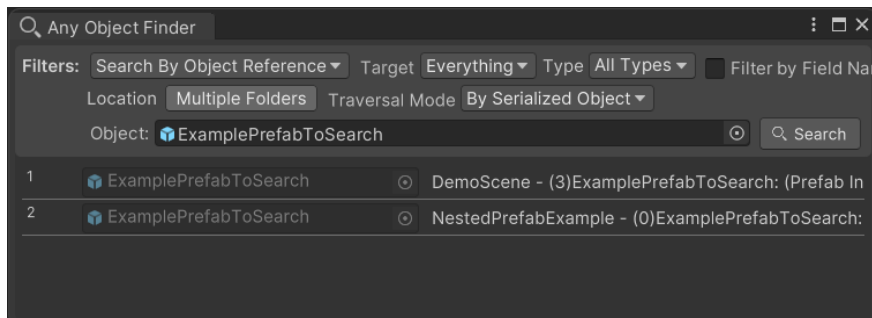
Search in all filtered objects' serialized property where it's a float with the matched condition. The arguments include the condition type (Nearly Equal, Less Than, Less Than Equal, More Than, More Than Equal, Not Equal) and a float value to compare.

d. Search by Boolean Value

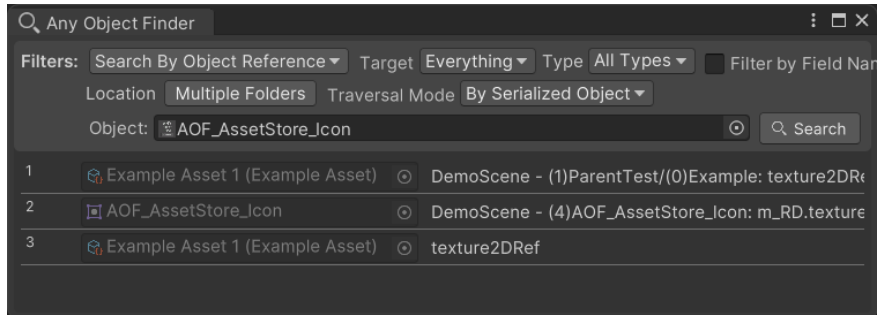
Search in all filtered objects' serialized property where it's a boolean with the same value as the specified Value.

e. Search by Enum Value

Search in all filtered objects' serialized property where it's an enum with a value name matching the text input. The argument is a string value, specifying whether it's case-sensitive or not.

f. Search by Object Reference

Search in all filtered objects' serialized property or prefab instance where it's a unity object with the same reference with the assigned object.



If we put an object (asset) other than a prefab, then it will search through all objects that refer to that specified object.

g. Custom Methods (extend by script)

We can add more custom methods, even with our custom arguments GUI by extending a script, which we will cover in the Customization chapter.

2. Targets (Can select multiple):

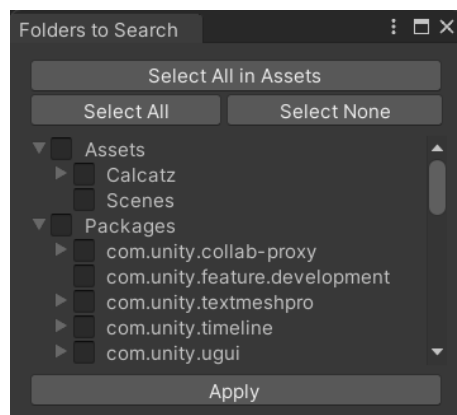
- **Scenes**
 - Search in scenes. The scenes to search will be from all scene assets inside the specified folder locations.
- **Assets**
 - Search assets in the specified folder locations.

3. Type

- All types
- or select a specific class derived from `UnityEngine.Object`.

4. Location

Select folders to search from both Assets and Packages.



5. Traversal Mode

The traversal method to use when iterating through all fields. The general idea is that we need to iterate through every public field and every field with *SerializeField* attribute.

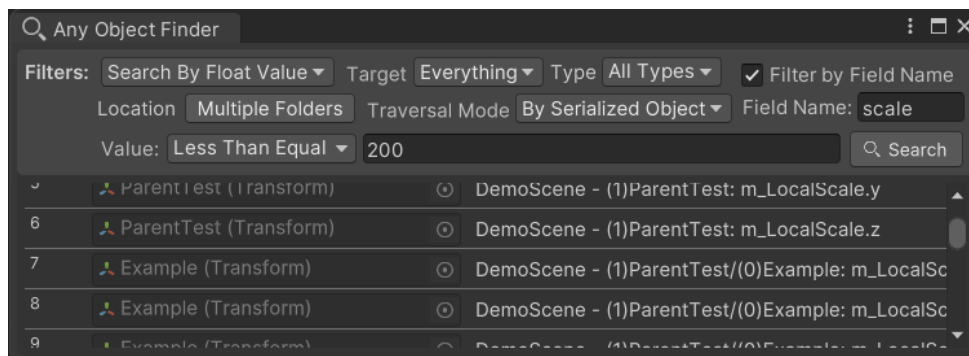
- **By Serialized Object**

Traverse all fields using Unity's *SerializedObject* by iterating through all *SerializedProperty*. This is the default traversal mode to use since it can easily find Unity's built-in fields which mostly are not serialized through C#'s attribute, but from native code.

- **By Reflection**

Traverse all fields using C#'s reflection. This is not recommended as is since it is less powerful to search for serialized fields since it can't detect built-in fields that are serialized natively. **But**, this can be developed in future versions for an ability to search through non-serialized fields that may be useful for runtime scanning.

6. Field Name



Normally, when searching for primitive fields, especially other than string, you'll get a lot more results than you expected. To narrow them down, check **Filter by Field Name**, and fill in the **Field Name** if you want to search within a specific field name only.

CUSTOMIZATION

Adding a custom search method can be done by creating a new class, derived from *BaseSearchBy*. Here's the list of the overridable methods:

1. *FilterObject*
 - The filter logic.
2. *CreateGUI*
 - The GUI area if additional parameter fields are needed.
3. *OnCreateResultItemGUI*

- The GUI template for each search result item. The usual one is to draw the object field in the left, and a text for the field's path in the right.

4. OnBindResultItemGUI

- Fill the contents of the previously created result item GUI with the actual result item data.

You may want to check the example script file called *MyCustomSearch* in which the purpose is to find objects that have a field with an odd integer value.



Here's the full script of the example:

```
#if UNITY_EDITOR

using Calcatz.AnyObjectFinder;
using System.Collections.Generic;
using UnityEditor;
using UnityEngine;
using UnityEngine.UIElements;

/// <summary>
/// A custom search-by example where it searches fields with odd integer value.
/// </summary>
public class MyCustomSearch : BaseSearchBy {

    // The filter logic
    public override List<ObjectFinder.ResultItem> FilterObject(ObjectFinder.TraversalMode traversalMode,
                                                                string fieldNameFilter,
                                                                Object rootObject,
                                                                Object objectToSearch,
                                                                SerializableSearchArguments searchArguments) {

        var resultItems = new List<ObjectFinder.ResultItem>();

        ComparerMethod<int> comparer = fieldValue => {
            return fieldValue % 2 == 1;
        };

        FieldFoundCallback<int> onFieldFound = (objectReference, fieldValue, fieldPath) => {

            if (rootObject != objectToSearch) {
                if (rootObject is SceneAsset && objectToSearch is Component component) {
                    fieldPath = GetSceneObjectPath(component) + ": " + fieldPath;
                }
            }

            if (resultItems == null) resultItems = new List<ObjectFinder.ResultItem>();
        };
    }
}
```

```
        resultItems.Add(new ObjectFinder.ResultItem() {
            targetObject = objectReference,
            info = rootObject == objectReference ? fieldPath : rootObject.name + " - " + fieldPath
        });
    };

    if (traversalMode.HasFlag(ObjectFinder.TraversalMode.BySerializedObject)) {
        TraverseSerializedObject<int>(fieldNameFilter, comparer, onFieldFound, objectToSearch);
    }
    if (traversalMode.HasFlag(ObjectFinder.TraversalMode.ByReflection)) {
        TraverseFields<int>(fieldNameFilter, comparer, onFieldFound, objectToSearch);
    }
    return resultItems;
}

// GUI area if additional parameter fields are needed.
public override void CreateGUI(VisualElement customAreaRoot, SerializedProperty serializedProperty) {
    base.CreateGUI(customAreaRoot, serializedProperty);
    customAreaRoot.Add(new Label("If additional parameters are needed, use this area.));
}

// GUI template for each search result item.
public override void OnCreateResultItemGUI(VisualElement parentElement) {
    base.OnCreateResultItemGUI(parentElement);
    parentElement.Add(CreateSelectableTextElement());
}

// Fill the contents of the previously created result item GUI with the actual result item data.
public override void OnBindResultItemGUI(VisualElement parentElement, int index, ObjectFinder.ResultItem resultItem) {
    base.OnBindResultItemGUI(parentElement, index, resultItem);
    if (parentElement.ElementAt(2) is TextField textField) {
        textField.value = resultItem.info;
    }
}

}

#endif
```

SUPPORT

If you found any bugs, or have suggestions related to this tool, you can send an email at affan@calcatz.com, or hit me up directly on Twitter [@dilaura_exp](https://twitter.com/dilaura_exp).